

PetriNetSimulate Documentation

Chris Dunne, MRes, MEng

December 9, 2024

PhD Student - CDT Future Propulsion and Power
NCCAT, Loughborough University
c.dunne@lboro.ac.uk

Contents

1	README	2
2	Component Nets and Phased Mission Logic	2
3	Main Algorithm	3
4	Adaptation for Repairable systems	4

Version History

Version 2.0 - 2024/12/02

- Simplified subnet definition with separate import matrices.
- Compressed A-matrices to support any numbering convention without increasing matrix size or computational cost.
- Enhanced parallelization handling for improved performance.

Version 1.0 - 2023/11/23

- Initial full release.

Version 1.0 - 2020/01

- First developed for Loughborough University Module TTD100 - Advanced Risk and Reliability Assessment.
-

1 README

V2: 2024-Dec-2nd Chris Dunne MEng, MRes

c.dunne@lboro.ac.uk

The following is a parallelised MATLAB code for simulating petri nets for a phased mission. Please note results have not been verified but appeared to be within expected limits and logic has been tested using the optional "opts.debugNetByPlotting = true" statement, which shows how tokens move through each net on a graph.

Connectivity between places is specified in 'A' matrices, and times to failure are generated from exponential/weibull/normal distributions. Data for these distributions is read in as stored as excel spreadsheets which are read in and assembled to a using the "AssembleAGlobal" function. All other parameters are specified at the start of the 'initialiser' scripts. Assumes all components active (could fail) during any phase (but will not cause phase failure if they are not in the phase petri net)

Numbering:

Numbering can be defined arbitrarily using the InitMarkingVariable which contains the indices of the component places which contain a token when the simulation starts. This can contain any arbitrary marking.

Disabling "Sim.CmptNetAlreadyInPhaseNet" in the setup script will cause the A-matrix to be constructed assuming that the places associated with component failure are simply the working places, offset by the number of components.

It is best practice, and easiest to account for things when 'component working' places are the first places to be defined (e.g. 1:10). Transitions to failed places are ideally numbered likewise. Components' failure places should then be the next integers following the working places (e.g. 11:19). Subsequent places in the net should be ordered methodically but order does not matter so long as A-matrix correctly programmed. (e.g. can begin at 100, after cmpt nets max out at 80)

Option of using innovative method of keeping component failure nets (2 places 1 transition each), separate from the main petri nets, allowing the failed nature of each component to carry across to all new phases - turn on Sim.TokenCopyingBetweenNets in the setup script.

Also built for subnets, which allow for additional repeated subnets to be created and used in multiple phases. These are imported as separate matrices and built into the global A-matrix automatically. Turn on Sim.TokenCopyingBetweenNets.

2 Component Nets and Phased Mission Logic

The code has three main inputs to simplify the creation of the 'global' A matrix. Phased missions require knowledge about which components have already failed to be stored when moving to the next phase. To handle this, this code has separated the 'component nets' from the rest of the A-matrix which is solved for. Component nets have two places and one transition per component.

If the phase times being too short creates an error, there is likely an issue with the "small" constant being too big for your phase duration. By default it's defined to be SimTime/1e5 but this may be too big if phases are considerably shorter than mission duration. Try making it smaller.

3 Main Algorithm

Algorithm 1 Phased Mission Petri Net Simulation

```
1: Initialize simulation parameters:
2: Load connectivity matrices, failure data, and configuration options.
3: Setup global matrices for places and transitions.
4: Initialize the marking vector for places ( $M_{0,Global}$ ) and failure tracking.
5: Setup parallel processing environment:
6: if ParallelSim then
7:   Initialize a parallel pool with specified number of processors.
8: end if
9: Simulation Loop:
10: for each simulation run (from 1 to  $MaxNSims$ ) do
11:   Initialize phase parameters:
12:   Set initial phase  $P = 1$ , system time  $t_{sys} = 0$ .
13:   Load phase-specific connectivity matrix and determine phase failure places.
14:   Generate failure times for components:
15:   Calculate failure times using specified distributions or random values.
16:   Loop until simulation end (endtime or sys failed)
17:   while mission not failed or successful do
18:     Check for failure
19:     if tokens reach failure places in current phase then
20:       Register system failure and exit loop.
21:     end if
22:     if  $t_{sys}$  exceeds current phase end time then
23:       if final phase then
24:         Mark simulation as successful and exit loop.
25:       else
26:         Increment phase:
27:         Update phase end time, reload connectivity matrices.
28:         Reset phase-specific variables.
29:       end if
30:     end if
31:     Identify enabled transitions:
32:     Evaluate tokens in input places of each transition to find all enabled transitions
33:     Find all transitions to fire by comparing enabled transitions with their remaining time
34:     Calculate time progression:
35:     Determine time increment  $\Delta t$  from minimum firing time for enabled transitions and .
36:     Increment system time and remaining transition times by  $\Delta t$ .
37:     Fire transitions:
38:     Update markings of places based on global phase connectivity matrix  $A$  and  $T_{fire}$ .
39:     Copy component net failed place tokens to corresponding phase net places
40:     Compute InsertionVector from component net to phase net connectivity
41:     Copy tokens
42:     Ensure each failed place token is only copied once per phase:
43:     Adjust AllowNetCopying to prohibit future copying of tokens that were just copied
44:     if debugging enabled then
45:       Plot the current marking of the Petri net for debugging.
46:     end if
47:   end while
48: end for
49: Post-simulation analysis:
50: Compute failure probabilities and component failure statistics.
51: Save results and generate visualizations.
```

4 Adaptation for Repairable systems

Code is already build for repairability in a single-phase (infinite phase duration) simulation. See example case file in code.

The code for a phased-petri net simulator is now adapted to repairability. Unlike a phased mission, which ends after the mission either fails or all phases succeed, repairability re-quires that after mission phases, a repair phase is added, or if there is no finite mission duration, these can be incorporated by adding repairability into the net and then declaring 'repairRateTable'.

The main principle is that you will be sim-ulating just 1 'simulation' but with a very large number of phases(working/maintenance/working...)($\sim 1e6$).

Repair times are coded to be constants, unlike failure times which are sampled from a distribution, so only mean time to repair need be declared.

Phased Repair

Repair phases must be treated differently and hence must first be identified. E.g. if $P=1:1e6$, and there are 2 different phases (working and maintenance), use $R = \text{mod}(P, 2)$ which will be 1 for repair phases, and 0 for normal phases. You could then declare only 2 A matrices, saving memory, one for failure and one for repair, and when loading APhase, load $A.\text{mod}(P,2)$ instead. If you have 2 working phases and 1 repair phase, you could use $R = (\text{mod}(P, 3) == 0)$ and a differnt A-matrix index.

The repair phase behaves quite differently from other phases and requires some modification in the code. It depends whether the repairability is modeled with revealed or unrevealed failures, and whether the pool of maintenance engineers is limited. The simplest case is where all failures are revealed at the end of the main mission, and all are repaired before the start of the next mission. Where some component failures are unrevealed, or only certain components are inspected if the mission didn't fail, additional modifications must be considered.

4.1 Approaches:

4.1.1 Component nets

These are used to keep track of component failure even after moving between phases. They could be useful if implemented properly for repair. Normally at the end of a phase we copy tokens from component net output places to phase net input places (*componentNetsToOutputPlaces*) and then prevent further copying for the rest of that phase. For repair, this still needs to happen at the start of the phase, but at the end of the phase, the component nets need to have their tokens moved from failed to working. This could be treated as a 'move' operation rather than a 'copy', since tokens are already in the component failed place. One must be taken out of that place and put into the working place.

If you assume all components are repaired before the next mission, there is a very simple option. Just reset the marking to the all working state at the end of a repair phase.

4.1.2 No component nets

The corresponding places in the A-matrix used directly, with tokens moving from failed to working places, through a 'in-repair' place in the repair phase net. This would however require a lot of modification to the code as it was always written around component nets but is very possible for simple cases. This is most easy if you do not need to know which components failed, just the total number. Otherwise, component nets may be easier.

4.2 Other important advice

In all cases you must remember that during a repair phase, the component times to failure must not be allowed to continue to tick down as if it were a normal phase. You can solve this by indexing $1:N_{\text{components}}$ where `tTimeRemaining` is updated, and not updating those transitions if `R` (`R` is true, where $R = \text{mod}(P, 2) == 0$).

Moreover, currently transition times are only initialised for a new simulation. This must be done after every repair phase in the loop now, using the `generateTimesToFailure` function. I recommend using a similar function to generate repair times even if they're all constants, for consistency and readability.

The main difference will be the condition you use for moving on to the next phase. That will now no longer be dependent on just phase end time but also on an 'all components repaired/inspected' condition if and only if phase is a repair phase.

You'll also need your phase times to be infinite if operating until failure, or repairing until all repaired.

If you have any pool of maintenance engineers, you'll have to declare that as a place and add the appropriate links in your A matrix, and set the place's marking to the number of engineers available at the start of every repair phase.